



Assignment 2: AWS Project

Individual Work: 20%

08.05.2026

Tasks

In this assignment, you will deploy an event-driven Python web application on AWS. The application allows users to upload PDF documents and compare two different text chunking strategies for retrieval. The retrieval uses a simple local TF-IDF embedding approach to avoid long latency caused by calls to LLM services and to avoid potential API rate-limit issues. You are required to submit a report describing your deployment and to attend a demonstration to verify the deployment.

Application Description

The application supports the following high-level workflow:

1. A user accesses the web application through a public web endpoint.
2. The user uploads a PDF document through the web application.
3. The uploaded PDF is stored in durable object storage.
4. The application records metadata about the uploaded document in a database.
5. The system processes the uploaded PDF using two different chunking strategies:
 - fixed-size chunking;
 - paragraph-aware chunking.
6. The system generates chunks and processing statistics for each strategy.
7. The web application retrieves processing status and generated results from the database.
8. Once both processing strategies have completed, the web application displays a side-by-side comparison of their results.
9. The user may enter a retrieval query to compare which chunks are retrieved by each strategy.

Required Architectural Properties

Your deployed system must satisfy the following architectural properties:

- The web application must be accessed through an Application Load Balancer.
- The web application tier must be deployed using an Auto Scaling Group. To minimise cost, the desired capacity may be set to 1.
- Uploaded PDF files must be stored in Amazon S3 rather than on the local file system of the web server.
- Application state, document metadata, processing status, and generated chunks must be stored in Amazon RDS.
- The system must decouple the upload request from PDF processing. The upload request should return without waiting for processing to complete.
- PDF processing must be performed by background worker processes rather than by the web application during the upload request. The system must include two worker processes, one for each chunking strategy: fixed-size chunking and paragraph-aware chunking. For simplicity, both worker processes may run on the same EC2 instance.
- The system must use Amazon SNS and Amazon SQS to decouple the web tier and the worker tier.
- Your deployment must use a custom VPC with appropriate public and private subnet placement.
- Public-facing and internal resources must be placed appropriately, and security group rules must be justified.

You may include other services and resources when appropriate. For instance, a NAT gateway may be needed to enable software installation on private instances.

You must make appropriate decisions about where to place resources in the network. In particular, you should consider which resources should be public-facing and which resources should remain private.

Setting up proper permissions is also essential for the workflow to function correctly. It is acceptable to set permissions slightly broader than strictly required for simplicity. However, you must acknowledge and justify this in your report.

Deployment Environment and Requirements

You are recommended to use AWS Learner's lab for this assignment. The Learner Lab provides a long-running environment with a \$50.00 credit and access to all the AWS services covered in this unit. Note that the credit is account based, if you have used learner lab before and spent some credit, you will not see a full \$50.00 credit this time. Also, there are specific usage limits for each service. Please read the documentation carefully and ensure that you stay within these limits.

Deliverable: Report (14 points)

Your submission must include a deployment report up to 10 pages including appendices. The report should document the architecture with details and justifications. It should also discuss potential extension and alternative architecture. The report must contain the following sections:

- **Introduction**

A brief introduction that provides context for the assignment. Avoid repeating the assignment specification verbatim. Instead, describe the overall purpose and highlight key architectural features in your own words.

- **Architecture Diagram (2 points)**

Provide a clear architecture diagram of your deployed system. The diagram should show:

- the VPC structure;
- public and private subnets, where relevant;
- the public entry point to the web application;
- the web application tier, including how web instances are managed or scaled;
- the EC2 instance or instances used to run the background worker processes;
- the storage used for uploaded PDF files;
- the event distribution mechanism used to trigger background processing;
- the message queues or equivalent job buffers used by the worker processes;
- the persistent database used for application state and processing results;
- the main interactions between components, including upload, event delivery, queue consumption, database updates, and result retrieval.

Your diagram should depict the actual AWS services you deployed, not generic labels only. For example, if your event distribution mechanism is implemented using a Simple Notification Service, the diagram should show SNS with icon similar to the AWS architectural styles used in AWS Academy labs. The diagram should be cover similar details as those used in the AWS Academy labs. The diagram should be prepared using a drawing software, such as draw.io. Hand-drawn diagrams will not be marked.

- **Web Application Deployment Description (6 points)**

Provide a concise description and justification of your AWS deployment. This section should focus on how the provided application is deployed, how the required AWS services are connected, and how access is controlled.

For each component below, describe its placement, key configuration, and relevant access control decisions, including important security group rules, IAM roles, or policies where applicable.

- **Web Tier and Load Balancer:** Describe how the web application is deployed using EC2, an **Auto Scaling Group**, and an **Application Load Balancer**. Include the VPC/subnet placement of the ALB and web instances, the basic listener and target group configuration, the ASG scaling policy and the security group rules that allow users to access the application through the ALB.
- **Worker Tier:** Describe where the worker instance is placed and how it is separated from the public web request path. Explain what access the worker instance needs in order to communicate with the required AWS resources and the database.
- **Database and Storage:** Describe how RDS and S3 are configured and used by the application. Include the network placement of RDS, whether it is publicly accessible, the security group rules controlling database access, and whether the S3 bucket is publicly accessible.
- **Event and Queue Resources:** Describe the event distribution and queueing resources used to connect the upload process to the background processing pipelines. Explain how the resources are connected at a high level and what permissions or policies are required for messages to be delivered and consumed.
- **Administrative Access:** Describe how you access and manage private resources, such as the worker instance or database, when needed. If you use a bastion host, explain its subnet placement, its security group rules, and how it reduces the need to expose internal resources directly to the public internet.

If any permissions or security rules are broader than strictly necessary, acknowledge this and briefly justify why you used them.

- **Evidence of Successful Deployment (1 point)**

Provide concise evidence that your system was working end to end before the report submission deadline. This may include selected screenshots of the deployed web application, key service configurations, monitoring information, or relevant logs.

You are not required to include many screenshots, as the demo will also verify the deployed system. Be selective and include only the most important evidence, as screenshots count towards the page limit.

- **Future Architecture Extension Discussion (2 points)**

The current deployment is designed for the scale and requirements of this assignment. In this section, propose how the architecture could be extended to support future requirements. You are not required to implement these extensions.

Your discussion should address **one or two** realistic future scenarios and explain how your architecture would change. Your discussion should include:

- what limitation or future requirement you are addressing;
- which AWS services or architectural components would need to change;
- any new permissions, configuration values, or external service credentials that would need to be managed;
- any trade-offs introduced by the proposed design.

- **Alternative Serverless Design Discussion (2 points)**

The current application uses message queues to decouple the web tier from the EC2 worker tier. An alternative design would be to use a serverless architecture in which the worker processes are replaced by Lambda functions.

Discuss whether Lambda would be suitable for this application. You are not required to implement the Lambda-based version. This section is intended to assess your ability to compare architecture alternatives and justify design choices.

- **Summary and Lessons Learned** Conclude your report with a concise summary of your key findings. We encourage you to reflect on any challenges you faced during the assignment and explain how you addressed them.

- **Report Style and Professionalism (1 point)**

Ensure your report is professionally written and well-organized with all required sections present. This includes using clear section headings, numbered figures and tables, and consistent formatting throughout. All AWS resources and components must be correctly named and referenced. AI usages and other references are acknowledged properly.

Deliverable: Live Demo (6 points)

In addition to submitting the deployment report, you must attend a live demonstration of up to 10 minutes to showcase the functionality of your application and your understanding of the deployed AWS architecture.

Assignment 2 demonstrations will be conducted in Week 14. To ensure your tutor can review your work beforehand, your report must be submitted at least 24 hours prior to your scheduled demo. Unless you have an extension longer than one week, your demo must be completed by 7:00pm Saturday (06/06/26) of Week 14. Your tutor will provide a booking link via Canvas and email. If you cannot attend your session, contact your tutor immediately; one reschedule may be permitted for valid reasons, provided it stays within the Week 14 window.

Your demoed deployment must match the architecture described in your report. If your report describes a resource, connection, permission, security group rule, or network placement that is not actually present in your deployed system, this may result in a loss of marks in the relevant report section.

During the demo, tutors may ask you to show AWS resources and explain how they correspond to your architecture diagram. For example, you may be asked to show the load balancer, Auto Scaling Group, EC2 instances, S3 bucket, event and queue resources, RDS database, subnet placement, security groups, IAM roles, and relevant application configuration.

Failure to attend the live demo will result in zero marks for the demo. In addition, because the report is intended to describe and justify the system that you actually deployed, failure to attend the demo means the teaching team cannot verify that the reported deployment exists and works as claimed. Therefore, if you do not attend the demo, the mark for the written report will be capped at 50%.

The demo will be marked using the following criteria:

- **End-to-end functionality (2 points):** You demonstrate that the application can be accessed through the load balancer, a PDF can be uploaded, the background processing workflow runs, and the web application displays the processing and comparison results.
- **AWS resource verification (2 points):** You show the key AWS resources used in your deployment and demonstrate that they match the architecture described in your report.
- **Explanation and understanding (2 points):** You answer questions about the architecture and individual services.

Application Codebase

You are provided with two versions of the code:

- Source code under the `mono` folder is a fully self-contained local web application. It uses an `in-memory SQLite` database for metadata and chunk data, and the local file system to store uploaded PDF files. No `event-driven architecture` is implemented, since all processing happens in a single process. The source code is contained in a single file, `app.py`. Instructions for setting up the environment and running the application can be found at the beginning of the code. This version is provided so that you can understand the behaviour of the web application. The interface is nearly identical to the interface of the AWS version of the application.
- Source code under the `aws` folder is an `AWS-ready version` of the application. The provided code expects you to set important AWS resource values, such as `DATABASE_URL`, `S3_BUCKET_NAME`, `FIXED_SIZE_QUEUE_URL`, `PARAGRAPH_AWARE_QUEUE_URL`, and other required values as environment variables. You will need to configure these values correctly based on the AWS resources you create. There are multiple source files for this version. A simple README file is provided to help you get started. As this version uses a number of Python libraries, it is important that you use `pip install -r requirements.txt` to install the specified package versions instead of installing the default latest versions.

Suggested Deployment Strategy

You are encouraged to develop and test your deployment incrementally. As a first step, you may run the `AWS-ready application` on a single EC2 instance to check that the application dependencies, environment variables, S3 access, database connection, and basic workflow are configured correctly.

This initial single-instance setup is only a development and debugging step. It does not satisfy the final architecture requirements of this assignment. Your final deployment must use the `required AWS architecture`, including the `Application Load Balancer`, `Auto Scaling Group`, `separate worker tier`, `event and queue resources`, `RDS`, `S3`, and appropriate VPC, subnet, permission, and security group configuration.

After confirming that the basic application works, you should progressively separate the components into the required architecture.

AI usage and proper acknowledgement

Use of Generative AI You may use generative AI tools to help you understand general concepts, debug deployment issues, and improve the clarity of your writing. However, your submitted report must describe your own deployment, configuration choices, screenshots, and understanding. You must not submit AI-generated text that describes resources, permissions, or behaviours that you did not personally implement or verify. All sources that you have consulted must be cited and referenced correctly. All instances of AI use also must be explicitly acknowledged.